

KCPC-CS130

Problem Sheet

Number Theory I: Primes, Structure, and Factorisation

Developed by the KCPC Internal Committee, this problem sheet follows the *Number Theory I* workshop. Problems are divided into three categories:

- Core practice set (to be attempted during the workshop)
 - Extended homework
-

I. Core Workshop Practice Set

These problems should be attempted during the workshop. They require careful modelling and correct algorithm selection.

For any questions that were not completed during the workshop, please attempt them independently afterwards.

Yersin's Note. The material that was necessary to be introduced was quite extensive, so we are forgoing integration into slides but will cover these in live walk-throughs towards the end of the workshop.

1. **Leetcode 204 - Count Primes**

Very simple sieve problem.

<https://leetcode.com/problems/count-primes/description/>

2. **AtCoder Beginner Contest 169 D - DivGame**

Simple factorisation question, effectively asks you to count the number of unique prime factors. Knowledge of the Fundamental Theorem of Arithmetic would be useful.

https://atcoder.jp/contests/abc169/tasks/abc169_d

3. **Project Euler 10 - Summation of Primes**

Very cute problem requiring use of sieves. Both linear and standard sieves can be used, and the speed-up is notable.

<https://projecteuler.net/problem=10>

4. **CSES 1082- Sum of Divisors**

Analytic problem. Do you notice anything about the sums of divisors? Think before you code

<https://cses.fi/problemset/task/1082/>

5. **CSES 1081- Common Divisors**

Nice question that involves basic arrays

<https://cses.fi/problemset/task/1081/>

6. **CSES 2182- Divisor Analysis**

Directly tests τ, σ, ϕ as mentioned in the lecture.

II. Extended Homework

Important. The following problems are intended to be extensive or difficult. They should only be attempted after completing all previously listed problems and assigned supplementary reading. They are intended as stretch challenges for those who wish to explore advanced modelling and optimisation techniques.

1. **CSES 1713 - Counting Divisors**

Textbook problem, asking students if they can recompute Euler totient function effectively.

<https://cses.fi/problemset/task/1713/>

2. **CodeForces 1498 C - Planar Reflections**

Extended question that can simplify nicely.

<https://codeforces.com/problemset/problem/1498/C>

3. **AtCoder Beginner Contest 215 D- Coprime 2**

Non-trivial coprime generation question. Any idea on how to simplify it?

https://atcoder.jp/contests/abc215/tasks/abc215_d

4. **CodeForces 687B - Remainders Game**

Can you implement Chinese Remainder theorem computationally? Imagine how hard this would be without modular arithmetic.

https://atcoder.jp/contests/abc215/tasks/abc215_d

5. **CodeForces 776E - The Holmes Children**

$g(n)$ actually has a very nice closed form. Can you find it?

https://atcoder.jp/contests/abc215/tasks/abc215_d

Guidance:

- Read constraints carefully before choosing an algorithm. An $\mathcal{O}(\sqrt{N})$ solution that works for $N \leq 10^{12}$ will TLE for $N \leq 10^{18}$.
- For sieve-based problems, decide upfront whether you need the SPF array or just a boolean `is_prime` array.
- Think before you code. Many of these problems have an analytic observation that makes the implementation trivial once found.
- Sanitize your modular arithmetic: negative results, overflow on intermediate products, and missing base cases are the most common sources of wrong answers.